01 THE JOURNEY

Can an open-source harness effectively self-improve?

A $100 development study — and the harder half turned out to be proving it.

HALF ONE — WHERE MOST WORK STOPS

Can a fixed open-weight model improve its own agent by rewriting only its prompt?

HALF TWO — THE REAL SUBJECT

…and can you actually measure whether it did?

The first half is the one everybody asks: can a model rewrite its own rulebook and get better?

The second half is the one almost nobody checks.

Most agent improvements are inside the noise.

228 scored runs · 192.9M tokens
github.com/cbarry10/LoopedBankingAgent

01 **The Journey**
02 Architecture
03 First Attempt
04 The Fixer Agent
05 The Results
06 Turn Counts
07 The Limit

02 ARCHITECTURE

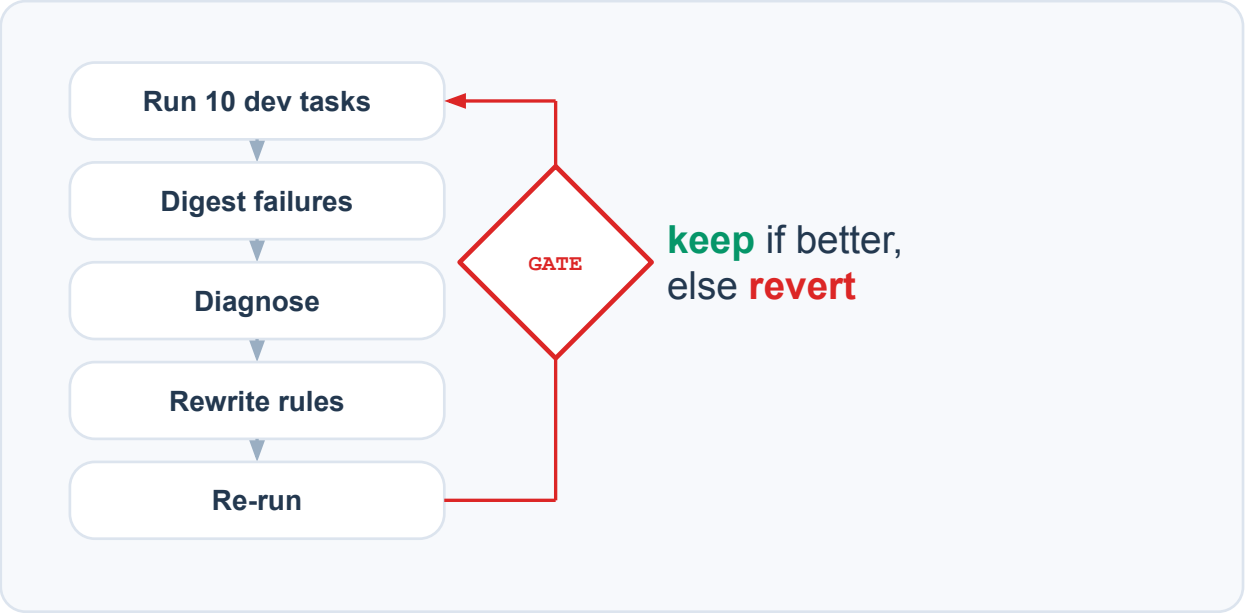
A looped “fixer agent” to learn from its mistakes.

The same model answers the task, plays the customer, and rewrites the rulebook between runs.

The benchmark scores the end-state of a database, not what the agent says. 97 tasks, 698 policy documents.

A run either ends because the customer stops, or because the agent runs out of steps.

The gate on the return path is where the story happens.



> Every arm shares one environment.
> Tools, knowledge base and scorer never change.

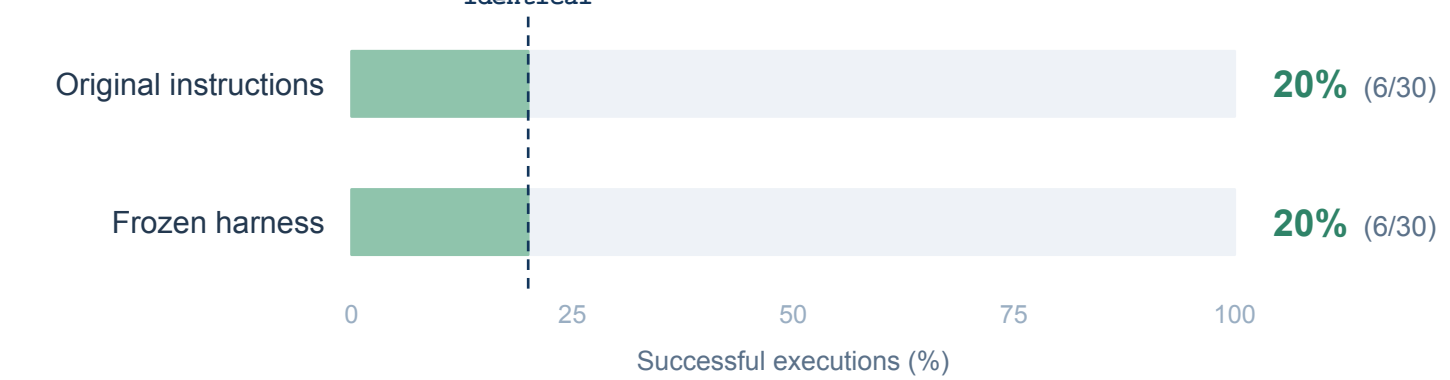
01 The Journey
02 **Architecture**
03 First Attempt
04 The Fixer Agent
05 The Results
06 Turn Counts
07 The Limit

03 FIRST ATTEMPT

The first attempt yielded no improvement.

Pre-registered, run on ten unseen tasks, and published before anything worked.

HELD-OUT RUN · 10 UNSEEN TASKS · SUCCESSFUL EXECUTIONS



HELD-OUT LIFT

+0

Identical reward and identical termination on all ten unseen tasks. Not close to zero — exactly zero.

01 The Journey
02 Architecture
03 First Attempt
04 The Fixer Agent
05 The Results
06 Turn Counts
07 The Limit

04 THE FIXER AGENT

Agents drafting their own .md files proved effective.

Three changes to how the loop measures and what it can see. Not one line of the rulebook.

THREE CHANGES — ALL TO THE METHOD

1. **Three trials per task**
   Reward becomes {0, .33, .67, 1.0} instead of pass/fail, so per-task luck averages out.
2. **A real keep margin**
   One full task equivalent, set above the measured noise floor.
3. **Full evidence**
   The ordered tool call → response trace, errors included, repeats flagged.

A human may author the *method*. A human never authors the *artifact under test*.

The fixer starts from an empty file.

Every line of the resulting rulebook is model-written. I changed how the loop measures, what it sees, and what it takes to keep a change.

KEPT VS REVERTED

1 keeper in **5** iterations

Four were reverted by the margin rule — including one I liked.

05 THE RESULTS

Successful runs doubled.

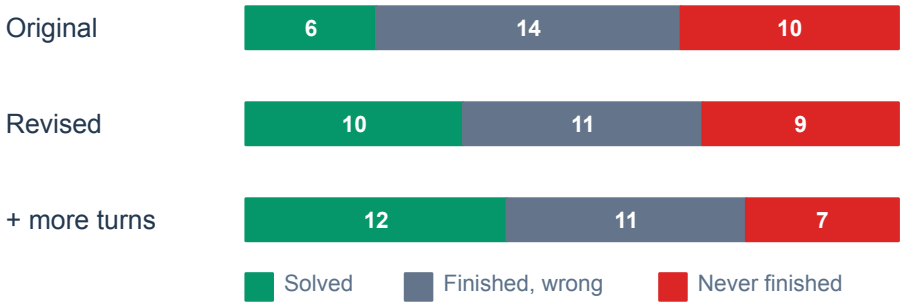
Revised instructions and more turns raised success from 6 to 12 executions out of 30.

Each configuration: 10 development tasks × 3 trials = 30 executions.

SUCCESS RATE



ALL OUTCOMES



COMBINED GAIN

+20 pp

+4 solved runs
From revised instructions

+2 solved runs
From additional turns

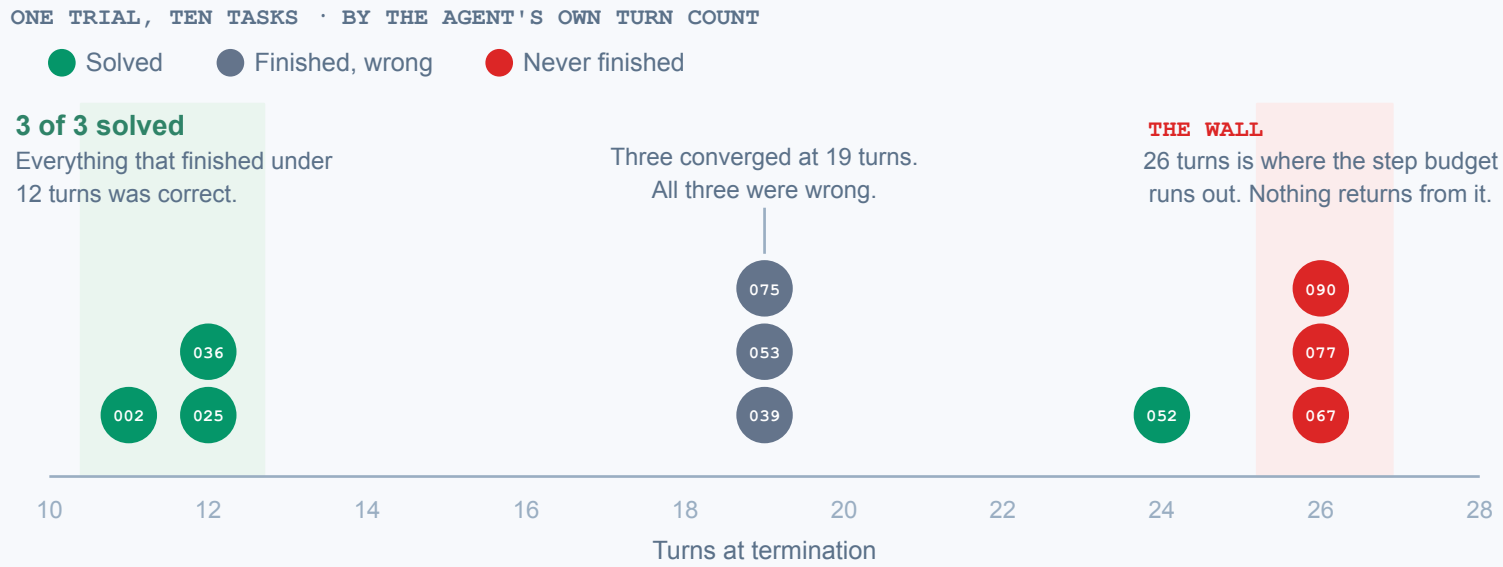
Development results.
The final version still needs a new held-out test.

01 The Journey
02 Architecture
03 First Attempt
04 The Fixer Agent
05 **The Results**
06 Turn Counts
07 The Limit

06 TURN COUNTS

Simple request were accurate. Multi-step tasks failed.

The best single trial: four solved out of ten, and the failure mode changes as turns climb.



The failure mode changes as turns climb. Under 12 turns the agent is correct. At 19 it still stops, but with the database wrong. At 26 it never stops at all.

One exception. task_052 solved at 24 turns — which is why this is a trend, not a rule.

Read as association, not cause. Turn count is partly a proxy for task difficulty. Extra turns bought nothing back.

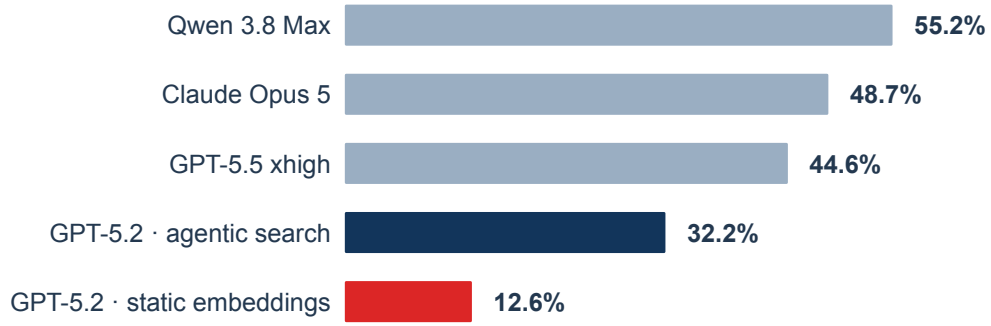
4 / 10	solved — the best trial
12	median turns when solved
22.5	median turns when not

07 THE LIMIT

What this result is not.

No absolute score is reported. Every number is a delta against a control run under identical conditions.

PUBLIC LEADERBOARD · SAME BENCHMARK FAMILY · CHECKED 2026-09-09



The same model swings 2.5× on retrieval strategy alone — a bigger gap than most models show against each other.

FOUR REASONS IT STAYS OFF THE BOARD

1. Tuning set — the 10 development tasks the method was optimised against.
2. Retrieval regime — BM25, top_k = 10.
3. Different simulated customer than every leaderboard row.
4. Different benchmark version.

Development results.
The 40% result has never been validated on held-out tasks — a fresh, untouched holdout is the next run.